

PigeOff: System Design Document

The following is a system design specification of the final project for the Internet-of-Things course at Sapienza University. The contributors are listed below:

Anja Škrlj	2285543
Filippo Zanei	2285059
Teun Boekholt	2284223

The GitHub repository is available at: <https://github.com/TeunBoekholt/Pigeoff>

1. Summary

The **PigeOff** system is an autonomous, IoT solution designed to protect historical monuments in Rome from structural degradation caused by the high acidity of pigeon excrement. The system detects pigeons using low-power LiDAR and computer vision, deploys localized deterrents within a critical time threshold, and uploads data over LoRaWAN for urban pattern analysis.

2. Design Constraints & Requirements

- **Operational Lifespan:** Minimum **7-day battery life** on a single charge cycle to map cleanly to municipal maintenance schedules.
- **Timing Windows:** The system must execute its entire sleep-wake-sense-classify cycle in under **10 seconds**. This is strictly bounded by the average pigeon stay duration of **16.3 seconds**, ensuring action is taken before defecation occurs.
- **Connectivity:** LoRa reporting to gateway at **30-minute intervals** using unconfirmed LoRaWAN uplinks to maximize battery longevity.

3. System Architecture

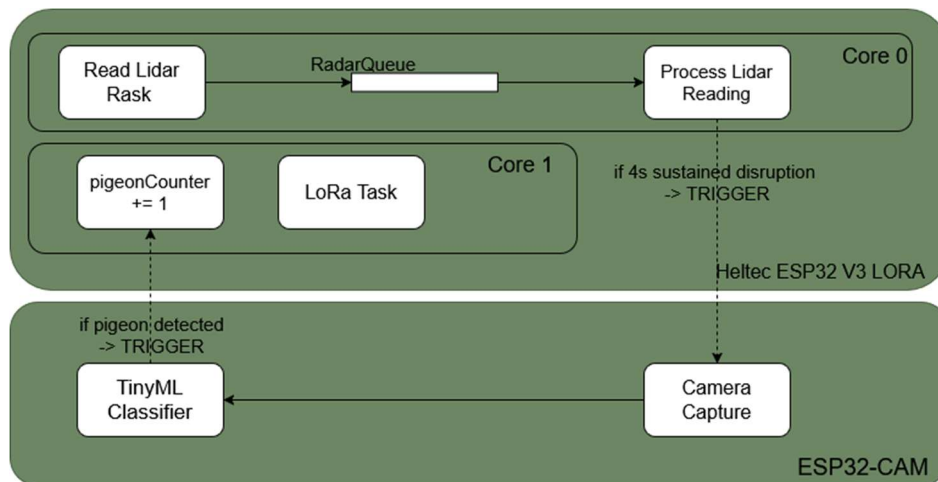
3.1 Components

- **Heltec ESP32 V3 with LoRa:** Connected to the LiDAR and responsible for processing LiDAR readings, triggering the ESP32-Cam when it detects movement and sending the pigeon count via LoRa every 30 minutes.
- **TF-mini LiDAR:** Small low-power LiDAR
- **ESP32-Cam:** Responsible for capturing the camera image and running the pigeon classifier. The two boards communicate via pin HIGH/LOW voltage triggers.
- **TTL-to-USB adaptor:** Necessary for uploading firmware to the ESP32-Cam

- **Power Supply**

3.2 FreeRTOS Architecture

The hardware architecture relies on 3 chips, with their tasks split as shown in the diagram:



3.3 Heltec ESP32 V3 with LoRa

3.3.1 LiDAR Reading and Processing

The LiDAR processing follows the following logic: there is a default reading, which is hard-coded into the board at device installation – this will be the LiDAR reading when the device is pointing at a clear statue. When the reading diverges from the default reading by 10cm (roughly the width of a pigeon), we call this sample “disrupted”.

We require 2 disrupted samples for the board to trigger the **ESP32-Cam** wake-up (e.g. a 4s sustained disruption). This number is chosen to comply with the 10s-time limitation while allowing a little over 5s for the inference, while also minimising power consumption. Requiring 2 disrupted samples, instead of making the sampling rate 0.25Hz, is a way of minimising false positives. Between samples, the board enters light sleep – except during active LoRa transmission.

3.3.2 LoRa Transmission

Every 15 minutes, we send via LoRa the number of pigeons detected in this time interval. The idea is to use this to analyse trends relating to pigeon activity. We are sending unconfirmed uplinks, to save energy. This is acceptable because missed samples are not critical for the sort of long-term analysis we expect to perform based on our data.

Note, that the board does not know the actual time on the clock, so these will not necessarily be sent at the full hour. For this reason, 15 minutes are chosen since it will allow reasonable robustness for the analysis of pigeon movement with respect to:

- Google Maps busy-ness data of a certain square
- Restaurant and cafe opening times

Additionally, 30 minutes allows the system to stay within the energy requirements.

During sending, a `LoralsActive` flag is maintained, preventing the LiDAR tasks from sending the board into sleep mode.

4. ESP32-Cam

When the LiDAR detects an object within target bounds, it wakes up the ESP32-CAM node, which takes a picture and runs inference. After this, it uses a pin voltage to send a trigger to the Heltec board if it has seen a pigeon. Since the whole logic is performed once at wake-up, all the code is in `setup()`.

4.1 Classification model

Due to computation constraints on the chip, we scaled down all images to 96x96 and transformed them into grayscale, drastically reducing the number of input features for our classifier. This decision was supported by our assumption that statues (the background of the images) would be white and that in relation to that the pigeons would be black.

The model used is a single layer Convolutional Neural Network followed by a dense layer for classification. The training dataset consisted mostly of black pigeon silhouettes against white backgrounds and as negative examples we showed it either completely white backgrounds or the silhouettes of branches and leaves. Although the size of the dataset was only 60 images, we used a built-in feature in Edge Impulse to artificially increase the amount of training data (by rescaling, mirroring the same image a few times), to still have a sufficiently large dataset.

The classifier is run locally by importing the library from Edge Impulse after training. The local classifier outputs a confidence score of seeing a pigeon. If it is above the threshold of 0.6 we put a Boolean value to true and the ESP-cam is instructed to pull its response pin to HIGH for one second, in order to give the ESP32-heltec time to receive the signal.

4.1 Image Capture and Preprocessing

We found the camera had some issues running the classification right after waking up compared to when it was just running constantly. In the end we attributed this to the fact that when the ESP-cam wakes up the camera needs to stabilize, e.g. the first few frames are otherwise just pure white because the autoexposure hasn't adjusted yet. To fix this, we delayed the camera about 1 second before taking a picture to run the inference. During this one second the ESP-cam takes three pictures in rapid succession and flushes them, to stabilize the autoexposure.

After taking the image, it is first pre-processed before passing it to the local classifier. The pre-processing step is simple, in that it only turns pixels to full black if their grayscale value is below a certain threshold – otherwise the pixel is turned to white. This way the images resemble most the training data of the classifier.